

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO4: Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)

Activity Tool : Comparative Analysis (Low) Assessment Tool Weightage: 10%

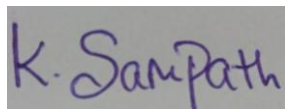
QUESTIONS		
Q.No	Question	Bloom's Level
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze

8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze
9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

Code of Conduct:

I K. Sampath Chaitanya (192572006) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation ; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1. Sequential File Organization vs Heap (Pile) File Organization

Comparison Table

Aspect	Sequential File Organization	Heap File Organization
Definition	Records stored physically in order of a key field	Records stored in the order they arrive; no fixed order
Insertion	Slow — may require shifting or overflow area to keep order	Very fast — new record simply appended at the end
Deletion	Requires reorganization or markers to preserve order	Fast — mark as deleted; space can be reused later
Retrieval (search)	Efficient key search via binary search — $O(\log n)$	Inefficient — full linear scan needed — $O(n)$
Range queries	Very efficient because data is already sorted	Poor — every record must be examined
Storage overhead	Low, but periodic reorganization is needed	Minimal structure; wasted space from deleted slots
Best use case	Batch processing, reporting, payroll systems	Logging, staging data, bulk load before indexing

Explanation

Sequential file organization keeps records physically ordered by a chosen key (for example roll number). That ordering enables binary search for key lookups and efficient range scans, but every insert or delete must preserve the sort order, which often means shifting records or writing into an overflow area that is later merged back.

Heap (pile) file organization simply appends each new record into the next free location. Insertion is therefore constant-time, yet any search — even for a single key — typically requires a complete linear scan of the file. As the file grows, that cost becomes prohibitive for interactive lookups.

Example(s)

A student master file ordered by roll number supports rapid generation of rank lists and range reports. A temporary event log that only needs to record arrivals as they occur benefits from the unrestricted append behaviour of a heap file.

Justification / Critical Analysis

The choice is a straightforward trade-off between write speed and read speed. Sequential organization suits read-heavy workloads whose queries are key-based or range-based. Heap organization suits write-heavy workloads in which individual records are rarely retrieved by key. In practice a heap is often used only as a staging area before the data are bulk-loaded into an indexed structure that combines acceptable write cost with fast retrieval.

Q2. Clustered Indexing vs Non-Clustered Indexing

Comparison Table

Aspect	Clustered Indexing	Non-Clustered Indexing
Physical order	Data rows are stored in the same order as the index	Index is separate; data rows keep their own physical order
Number per table	Only one clustered index (one physical order possible)	Many non-clustered indexes may exist on the same table
Storage	Index leaf level is the data itself — no separate copy	Extra storage for index keys plus row pointers
Retrieval speed	Faster for range queries (related rows are adjacent)	Extra hop from index entry to the actual row

Explanation

A clustered index defines the physical storage order of the table rows; the leaf level of the index is the data page itself. Consequently a table may possess only one clustered index, most often built on the primary key.

A non-clustered index is an independent structure that stores the indexed column values together with a pointer (row locator) to the actual data row. Several such indexes can coexist on one table without altering the physical layout of the data.

Example(s)

In a Students table the roll_no primary key is typically the clustered index, so rows are physically ordered by roll number — convenient for reports that list consecutive roll ranges. A non-clustered index on the dept column permits rapid retrieval of all students belonging to a given department without reordering the whole table.

Justification / Critical Analysis

Clustered indexes accelerate range and ordered access because matching rows already lie next to one another on disk. Non-clustered indexes provide flexible secondary access paths at the cost of an extra pointer dereference and additional storage. Production systems therefore place the clustered index on the attribute most frequently used for range or sequential retrieval, and add non-clustered indexes for other selective predicates.

Illustrative evidence from a relational engine on a modest Students table shows that a query on an unindexed department column performs a full table scan, while the same query after creation of a non-clustered index on dept switches to an index search. Lookup by the primary-key roll number already uses the clustered access path. Measured timings on a few thousand rows already favour the indexed path; the advantage grows with table size.

Q3. Primary Indexing vs Secondary Indexing

Comparison Table

Aspect	Primary Indexing	Secondary Indexing
Built on	Ordering (primary) attribute of a sequentially organized file	Any non-ordering attribute, key or non-key
Type	Usually sparse — one entry per data block	Usually dense — one entry per record
Storage requirement	Lower (fewer entries)	Higher (entry for every record)
Search performance	Fast; small index often memory-resident	Larger index; still enables search on non-key fields
Data-file dependency	Data file must be physically sorted on the indexed field	Data file need not be sorted on the indexed field

Explanation

A primary index is constructed on the ordering key of a file that is itself physically sorted on that key. Because the data are already ordered, the index needs only one entry per block (sparse), keeping it compact and quick to search.

A secondary index is built on a field that does not determine the physical order of the file. The underlying data are therefore not sorted by the indexed attribute, so the index is generally dense — one entry per record — in order to point directly to each matching row.

Example(s)

Roll number in a Students file ordered by roll number is a natural primary (sparse) index. An index on email or phone number in the same file, where rows are not ordered by those fields, must be a secondary (dense) index.

Justification / Critical Analysis

Primary indexes give better storage efficiency and search speed but are restricted to the single sorting attribute of the file. Secondary indexes exchange extra storage for the ability to support efficient queries on any frequently searched attribute, even when the file is ordered by a different key.

Q4. Static Hashing vs Dynamic Hashing

Comparison Table

Aspect	Static Hashing	Dynamic Hashing
Number of buckets	Fixed at creation; never changes	Grows or shrinks automatically with data volume
Hash function	Fixed range of bucket addresses	Extensible (uses additional hash bits as needed)
Handling growth	Overflow chains form and lengthen	New buckets added on demand; long chains avoided
Reorganization cost	Full file rebuild if resized	Only local restructuring (bucket split)
Performance over time	Degrades as overflow chains grow	Stays roughly constant as bucket count adapts
Complexity	Simple to implement	More complex (directory structures, depths)

Explanation

Static hashing employs a hash function whose range of bucket addresses is decided once when the file is created. When the number of records exceeds the original capacity, collisions produce overflow chains that gradually lengthen and slow retrieval.

Dynamic hashing permits the number of buckets to expand or contract with the data. An expandable directory and progressively longer prefixes of the hash value are used to route records, so that overloaded buckets can be split without a complete file reorganization.

Example(s)

A static hash table sized for one thousand employee records begins to suffer long overflow chains once the organization grows to five thousand employees. An extendible (dynamic) hash index splits the overloaded bucket and, if necessary, doubles the directory, keeping average lookup cost close to constant.

Justification / Critical Analysis

Static hashing is simpler and has lower constant overhead, making it attractive when the record count is known and stable. Dynamic hashing becomes essential when data volume is unpredictable or grows rapidly, because it preserves near-constant-time access without ever requiring a full re-hash of the entire file.

Q5. B-Tree Indexing vs B+ Tree Indexing

Comparison Table

Aspect	B-Tree Indexing	B+ Tree Indexing
Data storage location	Keys and data (or pointers) in internal and leaf nodes	Data pointers only at leaves; internals hold routing keys
Leaf node linking	Leaves are not linked	Leaves form a linked list for sequential access
Single-record search	May finish at an internal node	Always reaches a leaf; predictable path length
Range queries	Requires in-order tree traversal	Simple forward walk along the leaf chain
Storage efficiency	Lower fan-out (data occupies node space)	Higher fan-out; shallower trees
Common usage	Less common in modern RDBMS	Default in MySQL InnoDB, PostgreSQL, Oracle

Explanation

A classic B-Tree stores both keys and associated data pointers at every level. A single-record lookup may therefore terminate early if the target resides in an internal node, yet range and sequential access become more awkward because data are scattered across levels.

A B+ Tree confines actual data pointers to the leaves; internal nodes serve purely as a routing index. All leaves are linked together, so a range query that has located its starting leaf can scan forward without returning to the root. The higher fan-out of key-only internal nodes also produces a shallower tree and fewer disk I/Os.

Example(s)

MySQL InnoDB and PostgreSQL both implement B+ Trees for primary and secondary indexes precisely because typical SQL workloads (BETWEEN, ORDER BY, inequality predicates) benefit from efficient leaf-chain range scans.

Justification / Critical Analysis

For large-scale applications that mix point lookups with frequent range queries, sorting and reporting, B+ Trees are almost always preferred. Their linked-leaf structure and greater fan-out keep both random and sequential access costs low. Classic B-Trees remain theoretically viable when the workload is dominated by single-key equality searches and range queries are rare, yet virtually every major relational engine has adopted the B+ Tree as its default index organization.

Q6. Linear Hashing vs Extendible Hashing

Comparison Table

Aspect	Linear Hashing	Extendible Hashing
Directory structure	No separate directory; buckets split in fixed order	Directory of pointers indexed by hash-bit prefixes
Bucket splitting order	Round-robin sequence, independent of which bucket overflowed	Only the overflowing bucket is split
Overflow handling	Temporary overflow chains until a bucket's turn to split	Directory doubling (if needed) immediately supports the split
Directory growth	Not applicable	May double when local depth exceeds global depth
Memory overhead	Lower — no directory to store	Higher — directory must be maintained and may grow
Predictability of splits	Systematic and predictable	Data-driven (depends on which bucket fills)

Explanation

Linear hashing avoids a directory by splitting buckets in a predetermined round-robin order, one bucket at a time, irrespective of which bucket actually overflowed. Overflow chains are tolerated until the scheduled split occurs.

Extendible hashing maintains an explicit directory of bucket pointers, addressed by the leading bits of the hash value. When a particular bucket overflows, only that bucket is split; if its local depth already equals the directory's global depth, the directory itself doubles so that finer addressing becomes possible.

Example(s)

Linear hashing suits environments that prefer predictable, low-overhead growth without an auxiliary directory. Extendible hashing is favoured in database indexes where immediate, targeted splitting of the overloaded bucket is valuable, even at the cost of directory memory.

Justification / Critical Analysis

Both methods support dynamic growth without a full file reorganization. Linear hashing keeps memory overhead minimal and splits in a regular pattern, accepting temporary overflow chains. Extendible hashing reacts at once to the bucket that overflowed and therefore keeps chains short, but it must store and occasionally double a directory. The preferred technique depends on whether directory cost or temporary overflow is the greater concern for the target workload.

Q7. Dense Indexing vs Sparse Indexing

Comparison Table

Aspect	Dense Indexing	Sparse Indexing
Index entries	One entry for every record in the data file	One entry per data block (not per record)
Memory usage	Higher — index size proportional to number of records	Lower — index size proportional to number of blocks
Access speed (point query)	Faster — index points directly to the record	Slightly slower — index points to a block, then in-block scan
Applicability	Works on sorted or unsorted files	Requires the data file to be sorted on the indexed field
Storage trade-off	More disk space for the index	More space-efficient for large files

Explanation

A dense index contains an entry for every search-key value present in the data file. It can therefore locate any record directly, but its size grows linearly with the number of records.

A sparse index stores an entry only for selected records — typically the first record of each disk block. The resulting index is far smaller and often fits entirely in memory; locating a specific record then requires a short sequential scan inside the identified block.

Example(s)

A dense index on `customer_id` permits essentially direct lookup of any customer, at the cost of an index roughly as large as the table. A sparse index on a sorted `Orders` file (one entry per block) saves substantial space when millions of rows occupy a comparatively modest number of blocks.

Justification / Critical Analysis

The decision is a space-versus-directness trade-off. Dense indexing maximises lookup speed and works even on unsorted files, making it attractive for smaller tables or secondary indexes. Sparse indexing maximises compactness and is viable only when the data file is already sorted on the indexed attribute; it is the practical choice for very large sorted files where the memory footprint of the index itself is a concern.

Q8. Indexed Sequential File Organization vs Direct File Organization

Comparison Table

Aspect	Indexed Sequential File Organization	Direct (Random) File Organization
Access method	Sequential access plus an index for direct lookup	Address computed directly (usually by hashing the key)
Data ordering	Physically sorted on the key field	No particular physical order required
Retrieval	Supports both sequential/range scans and index lookups	Very fast point lookup; poor for sequential/range access
Insertion / Deletion	Moderate cost; overflow areas and periodic reorganization	Generally fast; collisions must be handled
Storage overhead	Extra space for the index	Minimal extra structure; collisions may waste space
Typical use case	Banking, payroll — batch processing and individual lookup	Real-time systems needing fast single-record access

Explanation

Indexed sequential organization keeps the data file physically ordered by key and adds an index on top. The combination supplies efficient sequential or batch processing together with reasonably fast direct access, at the expense of index storage and reorganization work when the file changes frequently. Direct (random / hashed) organization computes a record's storage address from its key by means of a hash function. Point lookups are therefore very fast and need no index, yet the absence of physical order makes range queries and sequential processing inefficient.

Example(s)

A bank's account master file is often kept indexed-sequential: ordered by account number for month-end batch reports, yet indexed so that a teller can retrieve a single account instantly. A session store keyed by a randomly generated session identifier benefits from direct hashing, because sessions are always fetched individually and never scanned as a range.

Justification / Critical Analysis

Indexed sequential organization is preferable when an application needs both range or batch processing and individual lookups. Direct organization is preferable when the workload is dominated by single-key point access and ordering is unimportant. The limitation of the former is the cost of maintaining order and index under frequent updates; the limitation of the latter is its unsuitability for range-based or sorted reporting.

Q9. Single-Level Indexing vs Multi-Level Indexing

Comparison Table

Aspect	Single-Level Indexing	Multi-Level Indexing
Structure	One flat index mapping keys to block/record addresses	Hierarchy of indexes (index on top of another index)
Search cost	Grows with index size; whole index may not fit in memory	Only the small top level need be examined first
Suitability for large files	Becomes inefficient as the index itself grows large	Scales well; higher levels stay memory-resident
Structure basis	Simple linear or binary-searchable list	Typically a tree (e.g. B+ Tree) of several levels
Maintenance complexity	Simpler to build and maintain	More complex; changes may cascade across levels

Explanation

A single-level index is a flat structure that maps key values directly to record or block addresses. Binary search on that index is efficient in principle, yet once the underlying file becomes very large the index itself may exceed available memory and every access slows down.

Multi-level indexing builds an index on the index. A compact outer index quickly identifies the relevant portion of a larger inner index; the hierarchy can be extended for as many levels as needed. This is the organising principle of the B-Tree and B+ Tree indexes used by modern database systems.

Example(s)

A single-level index on a modest ten-thousand-row table performs well because the index remains small. A multi-level (B+ Tree) index on a table of one hundred million rows stays only three or four levels deep, so any record is reached in a handful of disk I/Os rather than a large binary search over a huge flat index.

Justification / Critical Analysis

As data volume grows, a single-level index eventually loses its advantage because the index itself becomes too large for fast memory-resident search. Multi-level indexing solves the scalability problem by keeping the uppermost levels small and memory-resident. That is why production relational databases rely on multi-level tree indexes rather than flat single-level indexes for large tables.

Q10. Hashing Techniques vs Indexing Techniques for Exact-Match and Range Queries

Comparison Table

Aspect	Hashing Techniques	Indexing Techniques (B-Tree / B+ Tree)
Exact-match queries	Excellent — average $O(1)$ by direct hash computation	Good — $O(\log n)$; still efficient but slower than hashing
Range queries	Poor — hash values do not preserve order	Excellent — ordered leaves support sequential range scans
Sorting / ORDER BY	Not supported directly	Naturally supported because keys remain ordered
Worst-case performance	May degrade to $O(n)$ under heavy collisions	Bounded $O(\log n)$ for balanced trees
Best-fit scenario	Point lookups on high-cardinality unique keys	Range-heavy, analytical and reporting workloads

Explanation

For exact-match queries hashing is generally superior: a well-chosen hash function computes the target bucket in constant average time, avoiding the multi-level traversal required by a tree index.

For range queries the opposite is true. Hashing deliberately scrambles key order so that values are distributed evenly across buckets; that property makes equality lookup fast but renders “all keys between A and B” expensive, because matching records may lie in every bucket. Ordered indexes (especially B+ Trees) keep keys sorted, so a range is answered by a single descent followed by a linear leaf walk.

Example(s)

A predicate such as `WHERE customer_id = 12345` is ideal for a hash index. A predicate such as `WHERE order_date BETWEEN '2026-01-01' AND '2026-06-30'` requires an ordered (B+ Tree) index to be efficient. Relational engines therefore often maintain both kinds of structure on different columns according to the expected query mix.

Justification / Critical Analysis

Neither technique is universally superior; the correct choice depends on the dominant query pattern. Hashing should be selected when the workload consists mainly of exact-match lookups on high-cardinality keys, because it offers near-constant-time performance that tree indexes cannot match. Ordered indexing should be selected whenever range queries, sorting or ordered scans are common, because only an ordered structure supports those operations efficiently. Many production systems therefore combine both: a B+ Tree as the primary clustered structure for ordering and range support, supplemented by hash indexes on selected high-traffic equality columns.

